---------- Seaborn ----------

Seaborn is a Python data visualization library built on top of Matplotlib. It is used to create attractive and informative statistical graphs with less code.

Key Features :-

- Beautiful default styles (no need to customize much)
- Works directly with Pandas DataFrames
- Easy statistical plotting (mean, distribution, relationships)
- Built-in themes and color palettes

1. Line Plot

2. Scatter Plot

3. Strip Plot

4. Histogram

5. Regression Plot

6. Joint Plot

7. Bar Plot

8. Count Plot

9. Box Plot

10. Violin Plot

11. KDE Plot

12. Heatmap

13. Rug Plot

14. Pair Plot

15. PairGrid

In [ ]:

In [ ]:
```
# install seaborn & matplotlib
!pip install seaborn
```

In [8]:
```
# import seaborn
import seaborn as sns
# import matplotlib
import matplotlib.pyplot as plt
# import pandas
import pandas as pd
# import numpy as np
import numpy as np
```

In [9]:
```
# import dataset from github
penguins= sns.load_dataset("penguins")
penguins.head(10)
```

Out[9]:

	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g
0	Adelie	Torgersen	39.1	18.7	181.0	3750.0
1	Adelie	Torgersen	39.5	17.4	186.0	3800.0
2	Adelie	Torgersen	40.3	18.0	195.0	3250.0
3	Adelie	Torgersen	NaN	NaN	NaN	NaN
4	Adelie	Torgersen	36.7	19.3	193.0	3450.0
5	Adelie	Torgersen	39.3	20.6	190.0	3650.0
6	Adelie	Torgersen	38.9	17.8	181.0	3625.0
7	Adelie	Torgersen	39.2	19.6	195.0	4675.0
8	Adelie	Torgersen	34.1	18.1	193.0	3475.0
9	Adelie	Torgersen	42.0	20.2	190.0	4250.0

In [10]:
```
penguins["species"].value_counts()
```

Out[10]:
```
species
Adelie       152
Gentoo       124
Chinstrap     68
Name: count, dtype: int64
```

In [11]:
```
penguins["island"].value_counts()
```

Out[11]:
```
island
Biscoe       168
Dream        124
Torgersen     52
Name: count, dtype: int64
```

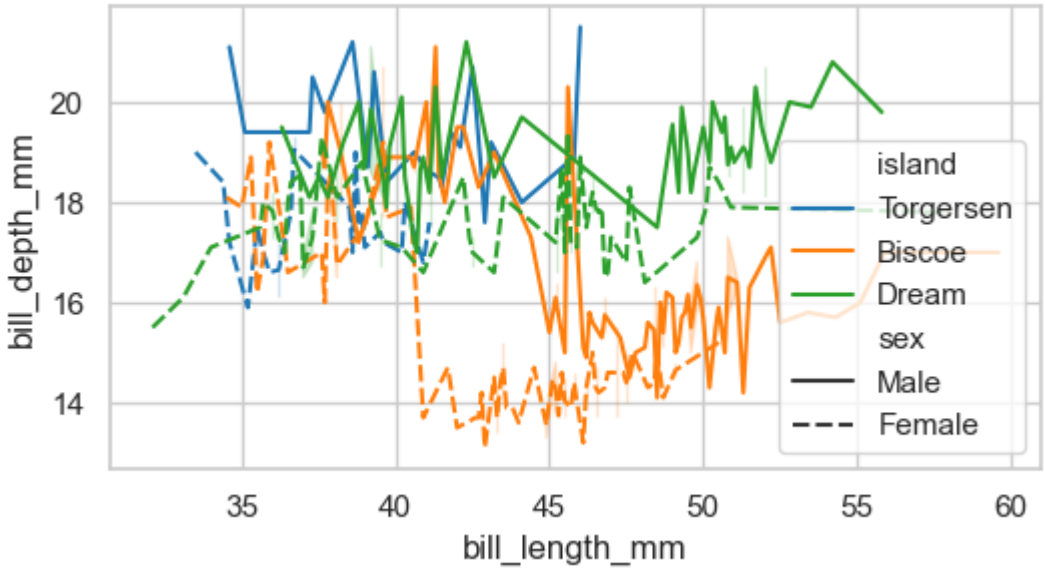
In [ ]:
```

```

1. Line plot

Shows trend over time or continuous data.

Points are connected with lines

In [77]:
```
plt.figure(figsize=(6,3))
sns.set_style('whitegrid')  # used for themes
sns.lineplot(x="bill_length_mm",y= "bill_depth_mm",data=penguins, hue='island',s
plt.show()
```

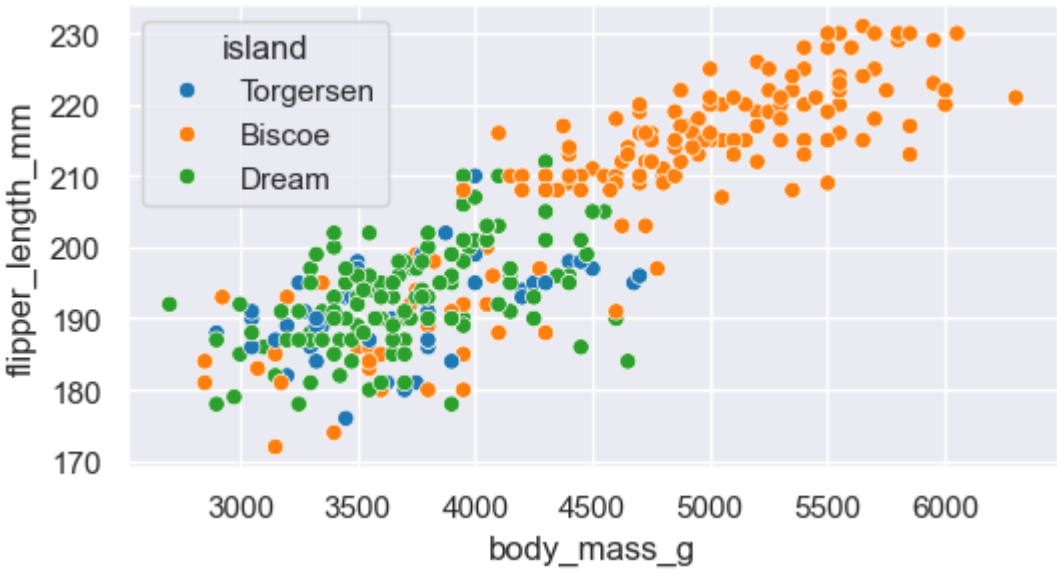


2. Scatterplot

Shows relationship between two numerical variables.

Displays individual data points

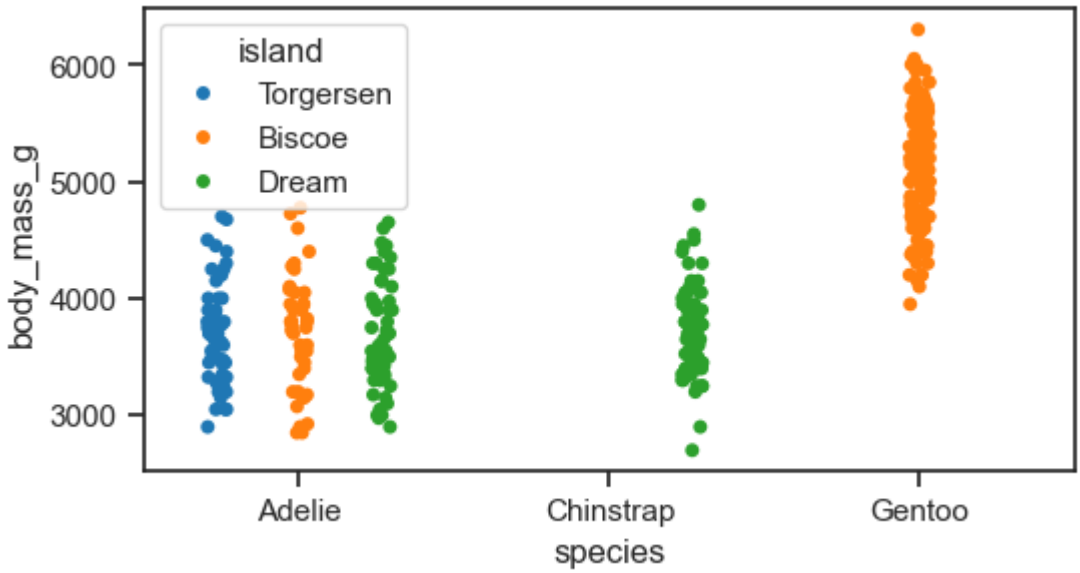
In [78]:
```
plt.figure(figsize=(6,3))
sns.set_style('darkgrid')  # used for background / themes

sns.scatterplot(x="body_mass_g", y="flipper_length_mm",data=penguins, hue= "isla
# hue is used to add an extra categorical variable to your plot using different

# sns.scatterplot(x="body_mass_g", y="flipper_length_mm",data=penguins, hue= "is

plt.show()
```



3. Strip Plot

A strip plot is used to display individual data points for a categorical variable. It shows how data is distributed across categories.

In [79]:
```
plt.figure(figsize=(6,3))
sns.set_style("ticks") # used for themes

sns.stripplot(data = penguins, x= 'species', y= "body_mass_g", hue = "island" ,
# dodge is a parameter used in categorical plots
# Separated (side-by-side) → dodge=True
# Overlapping (same position) → dodge=False

plt.show()
```

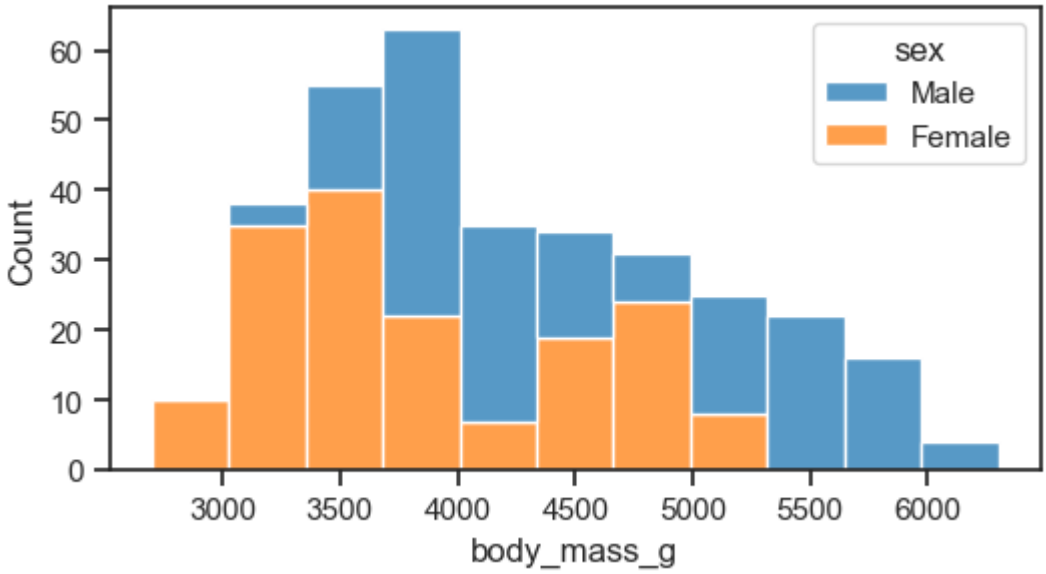


4. Histogram

Shows distribution of a single variable.

Data grouped into bins (frequency)

In [80]:
```
plt.figure(figsize=(6,3))
sns.set_context("notebook")
sns.histplot(x = "body_mass_g", data=penguins,hue="sex", multiple="stack")
plt.show()
```

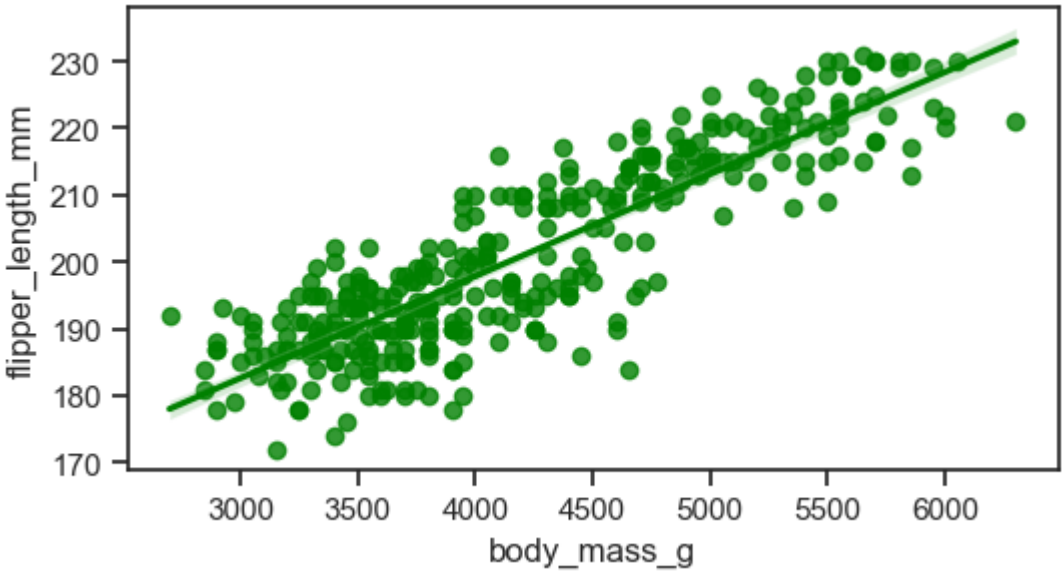


5. Regression Plot

A regression plot shows the relationship between two variables along with a best-fit line (regression line).

It combines: ▪ Scatter plot (data points) ▪ Regression line (trend)

In [81]:
```
plt.figure(figsize=(6,3))
sns.regplot(x="body_mass_g", y="flipper_length_mm", data=penguins, color = "gree
plt.show()
```



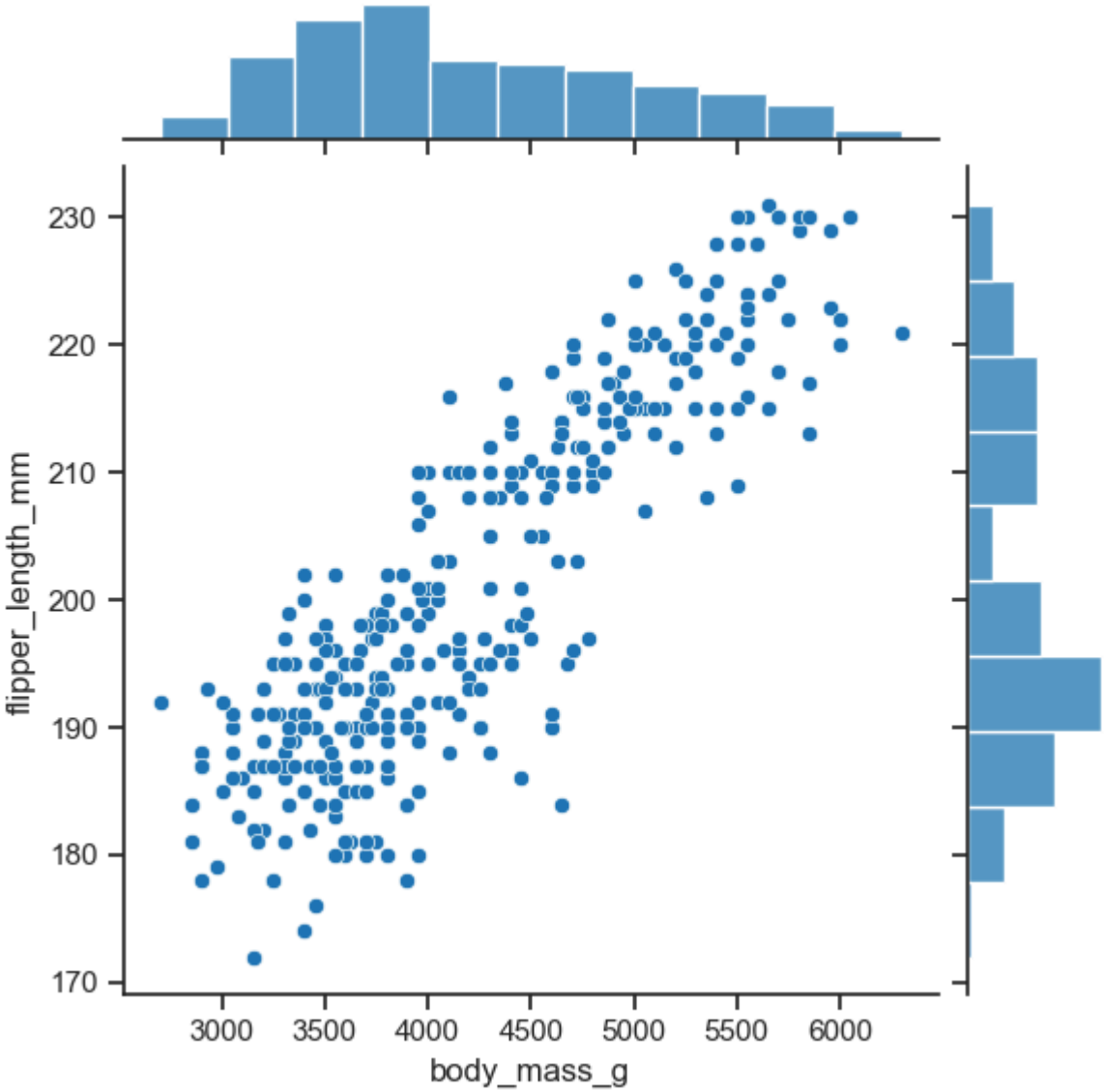
6. Joint Plot

Combines scatter plot + histograms

Shows relationship + distribution together

In [86]:
```
plt.figure(figsize=(6,3))
sns.jointplot(x="body_mass_g", y="flipper_length_mm", data= penguins)
# sns.jointplot(x="body_mass_g", y="flipper_length_mm", data= penguins,hue='sex'
plt.show()
```

```
<Figure size 600x300 with 0 Axes>
```

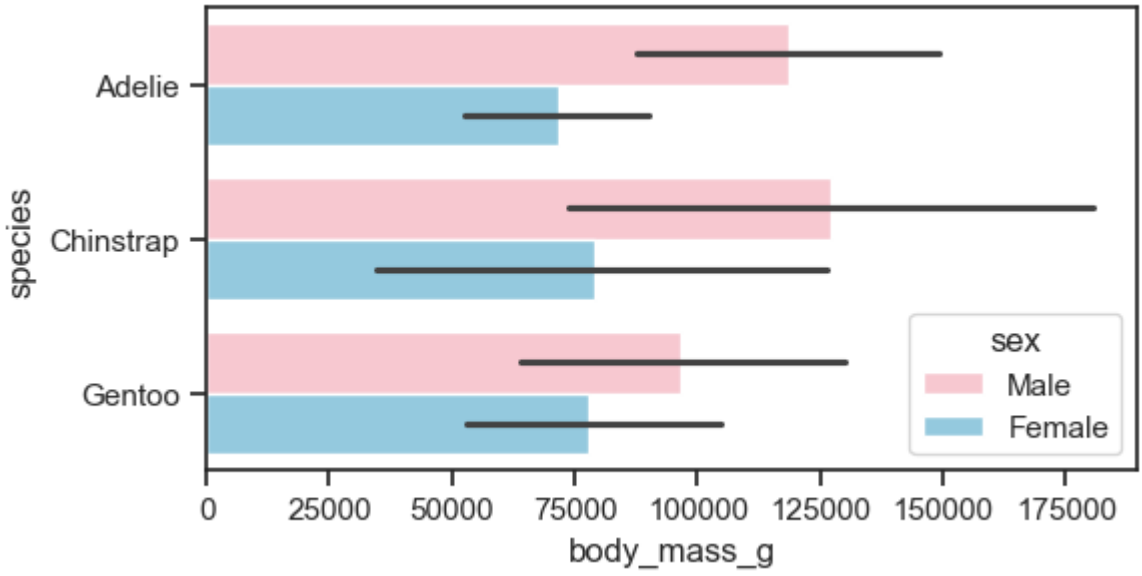


7. Bar Plot

Shows comparison between categories

Displays average (mean) values

In [87]:
```
plt.figure(figsize=(6,3))
sns.barplot(x="body_mass_g", y="species", data= penguins, hue = "sex", palette =
plt.show()
```



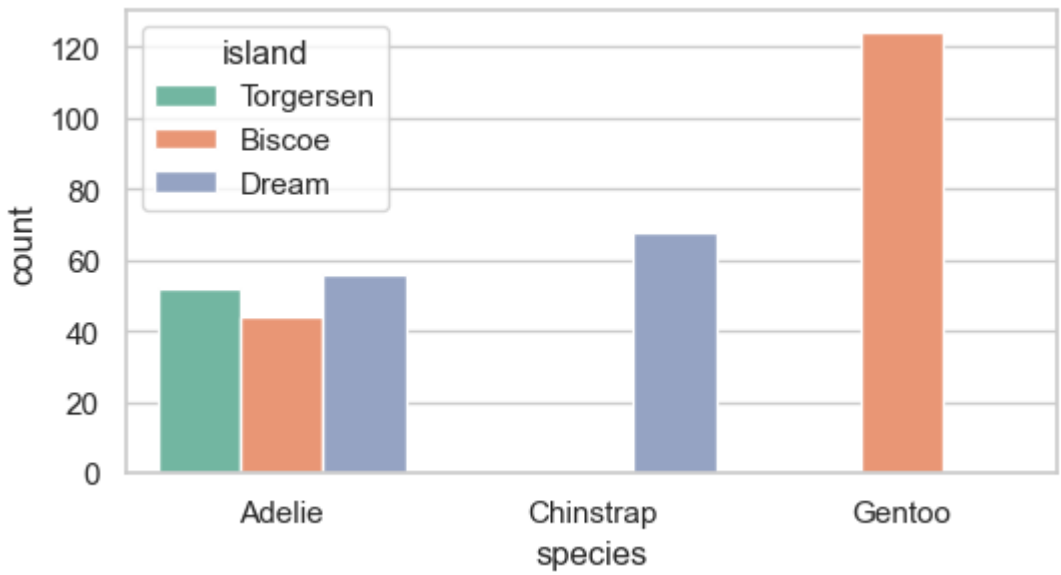
8. Count Plot

Displays number of occurrences of each category

Works on categorical data

Similar to a bar chart, but automatically counts values

In [88]:
```
plt.figure(figsize=(6,3))
sns.set_style('whitegrid')
sns.countplot(data = penguins, x = "species", hue = "island", palette = "Set2")
plt.show()
```



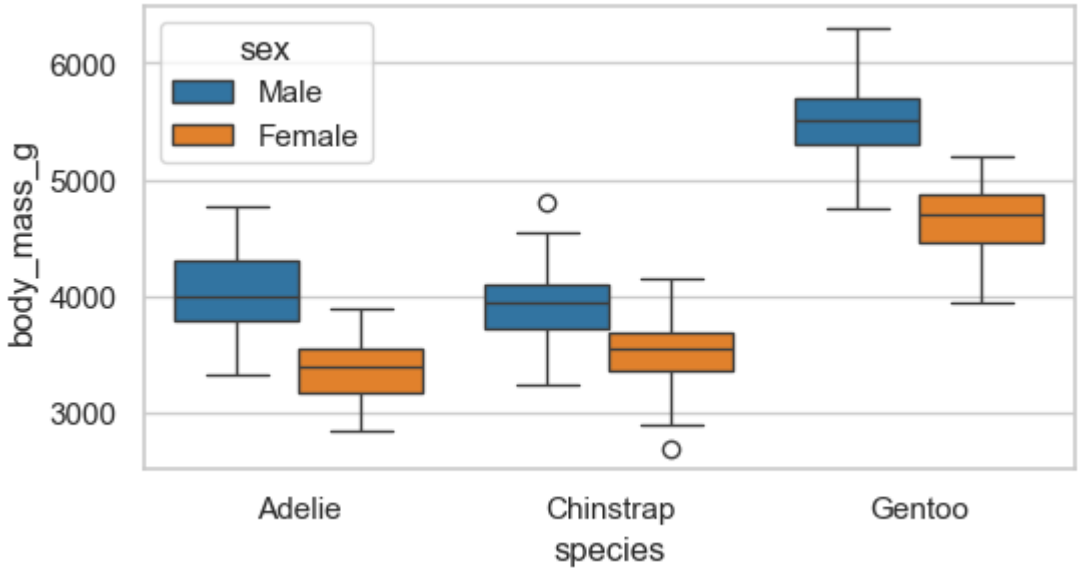
9. Box Plot

A box plot shows the distribution of numerical data using quartiles and helps detect outliers.

Key Points :-

- Shows median & quartiles
- Detects outliers easily
- Best for numerical + categorical data combination

In [89]:
```
plt.figure(figsize=(6,3))
sns.boxplot(data = penguins, x= "species", y= 'body_mass_g', hue="sex")
plt.show()
```



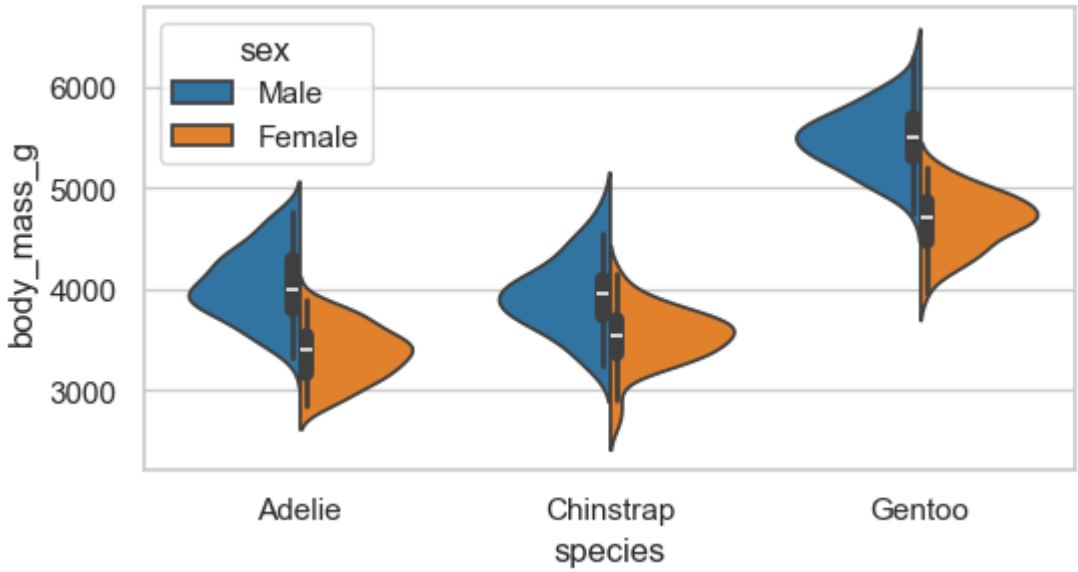
10. Violin Plot

A violin plot shows the distribution of numerical data and its density (shape). It is a combination of box plot + KDE (Kernel Density Estimation). Key Points :-

- Combines box plot + density plot
- Shows data shape clearly
- Good for comparing distributions

- Width of the violin: Density of the data at different values
- Middle line: Median
- Thick bar: Interquartile range (if inner: 'box')

In [90]:
```
plt.figure(figsize=(6,3))
sns.violinplot(data = penguins, x= "species", y= 'body_mass_g', hue="sex", split
plt.show()
```

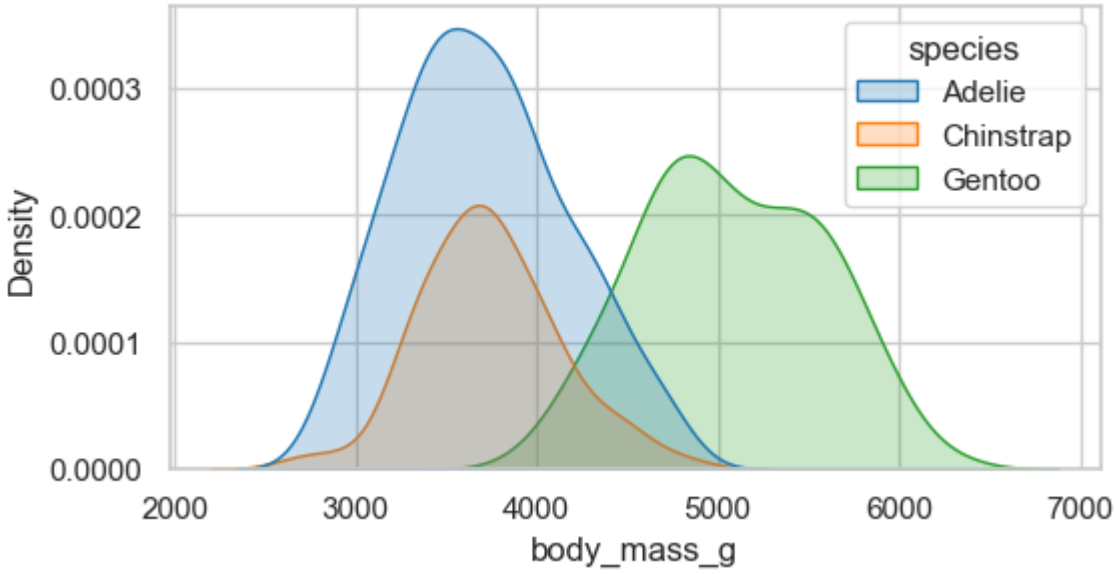


11. KDE Plot

A KDE (Kernel Density Estimation) plot shows the probability density (smooth distribution) of numerical data.

Key Points :- ▪ Smooth version of histogram ▪ Shows density, not count ▪ Good for continuous data ▪ Easy comparison using multiple curves

In [91]:
```
plt.figure(figsize=(6,3))
sns.kdeplot(data = penguins, x= 'body_mass_g', hue="species", fill = True)
plt.show()
```



12. Heatmap

A heatmap is used to visualize data in matrix form using colors.

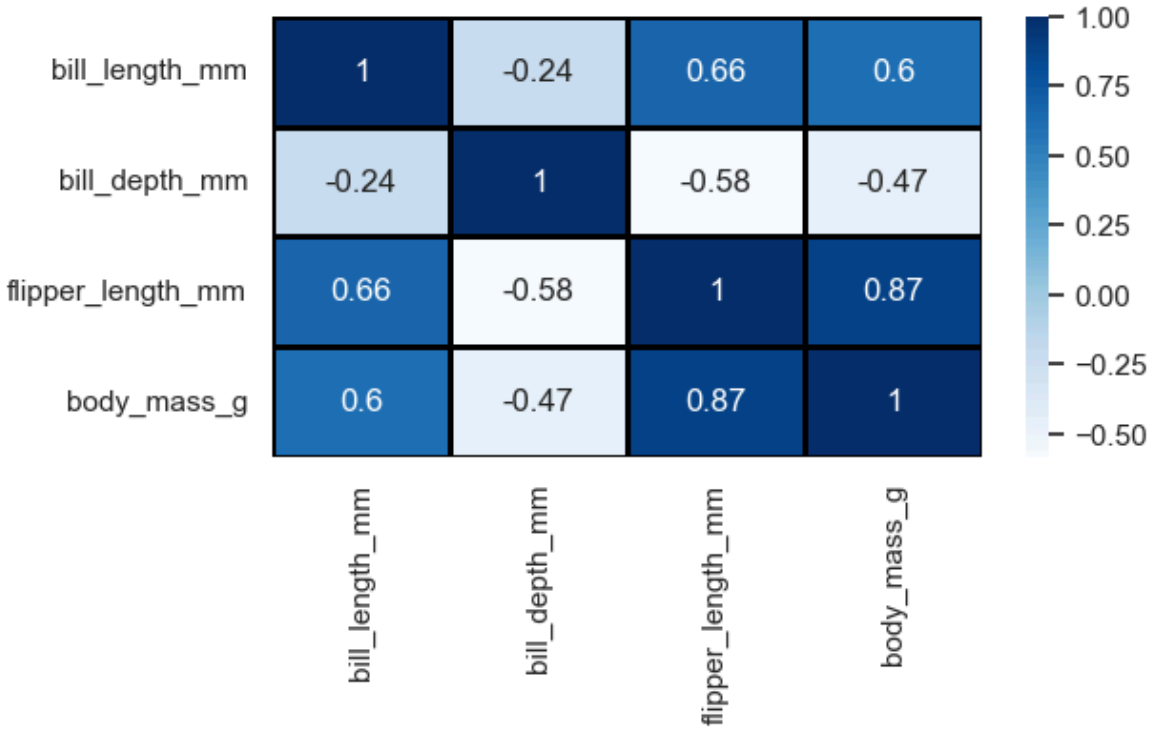
It helps quickly identify patterns, correlations, and intensity values.

In [38]:
```
columns =['bill_length_mm', 'bill_depth_mm', 'flipper_length_mm', "body_mass_g"]
penguins[columns].corr()
```

Out[38]:

	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g
bill_length_mm	1.000000	-0.235053	0.656181	0.595110
bill_depth_mm	-0.235053	1.000000	-0.583851	-0.471916
flipper_length_mm	0.656181	-0.583851	1.000000	0.871202
body_mass_g	0.595110	-0.471916	0.871202	1.000000

In [92]:
```
plt.figure(figsize=(6,3))
sns.heatmap(data=penguins[columns].corr(), annot=True , cmap="Blues", linewidth
# linewidth = outline on box
plt.show()
```

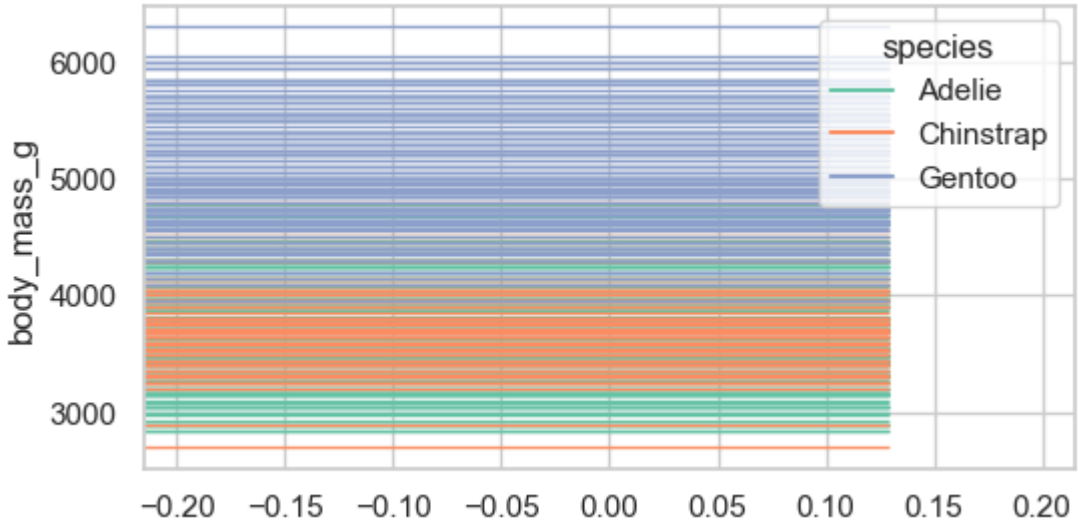


13. Rug Plot

A rug plot displays individual data points as small ticks on an axis.

It helps you see the exact distribution of data points.

In [93]:
```
plt.figure(figsize=(6,3))
sns.rugplot(data = penguins, y = 'body_mass_g' , hue = "species", palette = 'Set
plt.show()
```



14. Pair Plot

A pair plot shows relationships between multiple numerical variables in a dataset.

It creates a grid of plots (scatter plots + hist/KDE on diagonal).

In [95]:
```
sns.pairplot(data = penguins, hue ='sex' ,palette ='Set2', diag_kind = 'kde')
plt.show()
```

15. PairGrid

PairGrid is a more flexible version of pairplot.

It allows you to customize each plot in the grid manually.

In [72]:
```
graph = sns.PairGrid(data = penguins, hue= "sex", palette = "Set2")

graph.map_upper(sns.scatterplot)
graph.map_lower(sns.kdeplot)
graph.map_diag(sns.histplot)

graph.add_legend()

plt.show()
```

In [ ]:

In [ ]:

In [ ]:

In [ ]:

👉 The ways of styling themes / background are as follows :-

"white" , "dark" , "whitegrid" , "darkgrid" , "ticks"

👉 A palette is simply a collection of colors applied to your chart.

- Helps differentiate groups clearly
- Improves readability and aesthetics

"deep" , "muted" , "pastel" , "bright" , "colorblind"

"Blues" , "Greens" , "Reds" , "viridis" , "rocket"

"coolwarm" , "RdBu" , "Spectral"

👉 dodge is a parameter used in categorical plots

Separated (side-by-side) → dodge=True

Overlapping (same position) → dodge=False

👉 Seaborn provides 4 built-in contexts:

context controls the overall size and scaling of your plot elements like:
- text (titles, labels)
- lines
- markers

"paper" → small (for research papers)

"notebook" → default (balanced)

"talk" → larger (for presentations)

"poster" → very large (for posters)

In [ ]: